

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: COMPUTER ARCHITECTURE COURSE CODE: CSA12

COURSE OUTCOME COVERED

CO1: Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking (BL4)
Assessment Tool Weightage: 40%

QUESTIONS:5

Total Marks:25

ANNEXURE A APPLICATION BASED QUESTIONS

Code of Conduct:

I S Susmitha (192571068) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



ANNEXURE A

1. A smart city surveillance system processes continuous video feeds from multiple cameras, where large volumes of data are transferred between I/O devices, memory, and CPU. During peak hours, the system experiences lag and delayed response in threat detection. Analyze how the interaction between CPU, memory, and I/O contributes to this issue, and explain how different bus structures (single-bus vs multi-bus) and improvements in instruction execution cycle can enhance system performance.
2. A real-time stock trading application running on a processor executes millions of instructions per second, but users report delays in transaction processing during high load. The system has a high CPI due to frequent memory access and branch instructions. Examine how the fetch-decode-execute cycle impacts execution time in this scenario, and propose methods to reduce CPI and improve overall CPU performance using suitable architectural enhancements.
3. An embedded system based on an 8085 microprocessor is used in an industrial temperature control unit, where sensors provide input and actuators respond accordingly. However, incorrect temperature readings are occasionally processed due to improper use of addressing modes in the program. Discuss how different addressing modes influence data access, identify possible causes of such errors, and suggest how the instruction set of 8085 can be effectively used to ensure accurate and reliable operation.

4. A computing system uses an 8086 microprocessor with segmented memory for executing multiple programs simultaneously, but it encounters issues like incorrect data access and stack overflow errors. Evaluate how segmentation (code, data, and stack segments) works in 8086, analyze how improper segment handling can lead to such problems, and explain how instruction execution and addressing modes must be managed to ensure correct program execution.
5. A data communication system represents packet information in hexadecimal format for ease of debugging and transmission, but a software engineer mistakenly interprets values, leading to incorrect data processing. Analyze the importance of number system conversions between hexadecimal and binary in such systems, explain how errors in conversion can affect instruction execution, and discuss how efficient CPU design and benchmarking can help detect and minimize such issues in real-time systems.

MARKS ALLOCATION

Criteria	Understanding of Problem Context (20%)	Application of Concepts (20%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (20%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 1

Name	Register Number	Course Code	Course
S Susmitha	192571068	CSA1202	Computer Architecture

Question 1

A smart city surveillance system processes continuous video feeds from multiple cameras, where large volumes of data are transferred between I/O devices, memory, and CPU. During peak hours, the system experiences lag and delayed response in threat detection. Analyze how the interaction between CPU, memory, and I/O contributes to this issue, and explain how different bus structures (single-bus vs multi-bus) and improvements in instruction execution cycle can enhance system performance.

Understanding the Problem Context

A city-wide surveillance network generates a continuous, high-volume stream of video data from many cameras simultaneously. This data must move from I/O devices (camera interfaces/frame grabbers) into memory, and then to the CPU for analysis (e.g., motion or threat detection). During peak hours, the number of simultaneous I/O requests rises sharply, so the shared pathway connecting CPU, memory, and I/O becomes a bottleneck. Because threat detection is time-critical, any delay in this data path directly translates into a delayed real-world response, which is the core problem to be diagnosed.

Application of Concepts: CPU–Memory–I/O Interaction

In a typical von Neumann architecture, the CPU, main memory, and I/O controllers all communicate over a common set of address, data, and control lines. Video frames arriving from cameras must first be written into memory (often via DMA) before the CPU can fetch and process them. Three factors compound the lag during peak load:

- Contention for the bus: when multiple camera streams request memory access at once, only one transfer can use a shared bus at a time, so requests queue up.
- Memory access latency: high-resolution frames create large read/write bursts, and if memory bandwidth cannot keep pace with I/O arrival rate, buffers overflow or the CPU stalls waiting for data.
- CPU cycle stealing: without efficient DMA, the CPU itself must supervise data transfer, consuming cycles that would otherwise go to threat-detection computation.

Problem-Solving Approach: Single-Bus vs Multi-Bus Structures

Single-bus architecture: All units (CPU, memory, I/O) share one common bus for address, data, and control signals. It is simple and low-cost, but only one transfer can occur at any instant. Under peak camera load, this becomes the direct cause of lag, since video I/O transfers and CPU memory-fetches must wait their turn on the same line.

Multi-bus architecture: Separate or dedicated buses (e.g., a high-speed memory bus distinct from an I/O bus, or a hierarchical bus with bridges) allow the CPU to access memory while I/O transfers proceed concurrently on a different bus. This parallelism is essential for surveillance workloads, since it lets DMA-driven camera-to-memory transfers happen simultaneously with CPU-to-memory fetches for analysis, sharply cutting queuing delay during traffic spikes.

For this scenario, migrating from a single-bus to a multi-bus (or bus-hierarchy with a dedicated high-bandwidth memory bus) design directly addresses the bottleneck, because it removes the single point of contention between simultaneous video-ingesting and processing traffic.

Improving the Instruction Execution Cycle

The fetch–decode–execute cycle itself can be optimized to help the CPU keep pace with incoming data:

- **Pipelining:** overlapping the fetch, decode, and execute stages of successive instructions increases instruction throughput, so more frames can be analyzed per second.
- **Efficient use of DMA:** letting I/O controllers transfer data directly to memory without CPU intervention frees CPU cycles purely for detection algorithms rather than data shuffling.
- **Cache utilization:** keeping frequently accessed detection-model data and recent frame buffers in cache reduces the number of slow main-memory accesses per instruction cycle.
- **Interrupt-driven I/O over polling:** interrupts let the CPU perform useful work between data arrivals instead of continuously polling camera ports, reducing wasted cycles during peak load.

Solution Summary and Justification

Combined, a multi-bus (or bridged bus-hierarchy) structure resolves the bus-contention bottleneck by allowing concurrent I/O and CPU-memory transfers, while pipelining, DMA, caching, and interrupt-driven I/O shorten and better utilize each instruction execution cycle. Together these measures increase effective throughput and reduce the queuing delay that causes lagged threat detection during peak hours, giving the system the low-latency response it requires.

Question 2

A real-time stock trading application running on a processor executes millions of instructions per second, but users report delays in transaction processing during high load. The system has a high CPI due to frequent memory access and branch instructions. Examine how the fetch–decode–execute cycle impacts execution time in this scenario, and propose methods to reduce CPI and improve overall CPU performance using suitable architectural enhancements.

Understanding the Problem Context

A stock trading platform must execute a huge number of instructions per second with strict latency requirements, since even small delays can cause missed trades. The reported delay under high load, combined with a high Cycles Per Instruction (CPI), indicates that although the instruction count is high, each instruction is consuming more clock cycles than ideal — largely due to frequent memory accesses (fetching prices, order books) and branch instructions (conditional trading logic, order validation checks).

Application of Concepts: Execution Time and the Fetch–Decode–Execute Cycle

Execution time is governed by the fundamental performance equation:

$$\text{Execution Time} = \text{Instruction Count} \times \text{CPI} \times \text{Clock Cycle Time}$$

In the naive fetch–decode–execute cycle, each instruction passes through its stages sequentially: the instruction is fetched from memory, decoded, and then executed (which may itself require a further memory access for data). If memory access is slow relative to the CPU clock, the fetch and memory-operand stages dominate cycle count, inflating CPI. Similarly, every branch instruction disrupts the sequential flow of this cycle: the CPU cannot know the next instruction to fetch until the branch is resolved, so cycles are wasted waiting for that resolution or worse, an incorrectly assumed instruction stream must be discarded and re-fetched.

Problem-Solving Approach: Reducing CPI

- 1. Instruction pipelining:** Overlapping the fetch, decode, and execute stages of consecutive instructions allows several instructions to be in different stages simultaneously, reducing the effective cycles consumed per instruction and directly lowering CPI.
- 2. Memory hierarchy and caching:** Introducing or enlarging L1/L2 caches for frequently accessed price data and order structures reduces the number of slow main-memory accesses, since cache hits complete in far fewer cycles than main-memory reads.
- 3. Branch prediction and speculative execution:** A branch predictor estimates the likely outcome of conditional trading logic (e.g., buy/sell threshold checks) so the pipeline can continue fetching down the predicted path instead of stalling; this substantially reduces cycles lost to branch instructions.
- 4. Reducing pipeline hazards:** Techniques such as forwarding/bypassing (to resolve data hazards) and delayed branching or branch-target buffers (to resolve control hazards) keep the pipeline full and prevent stalls.
- 5. Superscalar / multiple-issue execution:** Issuing more than one instruction per clock cycle, where dependencies allow, increases throughput and lowers effective CPI further for high-instruction-count workloads like trading engines.

Solution Summary and Justification

Since Execution Time depends directly on CPI, the fixes that reduce cycles wasted on memory stalls (caching, pipelining) and on branch resolution (prediction, hazard reduction) are the most effective architectural enhancements here. Applying pipelining together with a well-tuned cache hierarchy and branch prediction directly targets the two causes named in the scenario — frequent memory access and branch instructions — and would

measurably lower CPI, shortening execution time and eliminating the high-load transaction delays experienced by users.

Question 3

An embedded system based on an 8085 microprocessor is used in an industrial temperature control unit, where sensors provide input and actuators respond accordingly. However, incorrect temperature readings are occasionally processed due to improper use of addressing modes in the program. Discuss how different addressing modes influence data access, identify possible causes of such errors, and suggest how the instruction set of 8085 can be effectively used to ensure accurate and reliable operation.

Understanding the Problem Context

In this embedded temperature-control unit, the 8085 microprocessor must repeatedly read sensor values (input), compare them against set thresholds, and drive actuators accordingly. The described fault — occasionally incorrect readings being processed — points to the program accessing the wrong memory location, register, or stale data, rather than a hardware sensor fault. Since addressing modes determine exactly how and from where an instruction's operand is fetched, an error in choosing or coding the addressing mode is a very plausible root cause.

Application of Concepts: Addressing Modes in 8085 and Their Influence on Data Access

Immediate addressing: the operand is a constant specified within the instruction itself (e.g., MVI A, 32H). Useful for fixed threshold values, but if a threshold needs to change dynamically, hard-coding it via immediate addressing would cause the program to keep comparing against an outdated constant.

Register addressing: the operand lies in a CPU register (e.g., MOV A, B). Fast, but if the wrong register is referenced due to a coding slip, the processor would compare or output the wrong quantity — for instance, an old, previously read value instead of the current sensor input.

Direct addressing: the instruction specifies the exact 16-bit memory address of the operand (e.g., LDA 2050H). If the address constant is mistyped or if the sensor's latest value is actually stored elsewhere due to a buffering error, the CPU will load a stale or unrelated memory cell as the 'current' reading.

Register indirect addressing: the address of the operand is held in a register pair (e.g., LDAX B, or MOV A, M using the H-L pair). This is commonly used for reading sequential sensor buffers; if the pointer register pair is not updated or incremented correctly, the program can repeatedly read a stale or wrong buffer slot.

Indexed/implicit addressing: used implicitly by certain instructions (e.g., flag-based conditional jumps). Misuse — such as failing to check the correct status flag after a comparison — can cause the control flow to process an invalid reading as valid.

Because each addressing mode fetches its operand from a structurally different source (instruction stream, register, or memory), a mismatch between the intended data source and the addressing mode actually coded is a direct and common cause of incorrect data being processed.

Problem-Solving Approach: Identifying Likely Causes

- A pointer register pair (H-L or B-C) used for register-indirect addressing not being refreshed before each read, causing the same or an unrelated memory location to be reread as if it were the newest sensor sample.
- A direct-addressed memory location for the sensor buffer being hard-coded incorrectly, or overlapping with another variable's memory cell, corrupting the value read.
- Use of immediate addressing for what should have been a variable threshold, so comparisons are always made against a fixed rather than updated value.
- Missing or incorrect flag checks after arithmetic/compare instructions (e.g., not verifying the Carry or Zero flag correctly), leading the program to branch on stale condition results.

Suggested Use of the 8085 Instruction Set for Reliable Operation

- Use register-indirect addressing (via the H-L pair, e.g., MOV A, M / LDAX) for reading sequential sensor buffer values, ensuring the pointer is incremented (INX H) after every read so each cycle accesses fresh data.
- Reserve immediate addressing strictly for genuinely fixed constants (e.g., calibration offsets), and use direct or indirect addressing for any quantity — like thresholds — that may be updated at runtime.
- Employ compare instructions (CMP, CPI) followed by explicit conditional jump instructions (JZ, JC, JNC) to make the decision logic for actuator control transparent and directly tied to the current comparison result.
- Use I/O instructions (IN/OUT) consistently for sensor and actuator ports so the addressing of peripheral devices is unambiguous and separate from general memory addressing, reducing the chance of confusing memory and I/O locations.

Solution Summary and Justification

The occasional incorrect readings are best explained by a mismatch between the addressing mode used and the actual location of current sensor data — most often a stale pointer in register-indirect addressing or an incorrectly fixed immediate value. Correcting the program to use indirect addressing with proper pointer updates for buffered sensor data, and reserving immediate addressing only for true constants, together with disciplined use of compare-and-branch instructions, ensures the 8085-based controller consistently accesses and acts on accurate, current temperature data.

Question 4

A computing system uses an 8086 microprocessor with segmented memory for executing multiple programs simultaneously, but it encounters issues like incorrect data access and stack overflow errors. Evaluate how segmentation (code, data, and stack segments) works in 8086, analyze how improper segment handling can lead to such problems, and explain how instruction execution and addressing modes must be managed to ensure correct program execution.

Understanding the Problem Context

The 8086 uses a segmented memory model to address 1 MB of memory using 16-bit registers, by dividing memory into logical segments for code, data, stack, and extra data. When multiple programs run together, each needs its own set of segment boundaries; incorrect data access and stack overflow errors suggest that segment registers or offsets are being managed incorrectly, causing one program's logical addresses to spill into another's memory region, or a stack segment to run out of reserved space.

Application of Concepts: How Segmentation Works in 8086

The 8086 computes every physical address as $\text{Segment Register} \times 10\text{H}$ (shifted left 4 bits) + Offset, giving a 20-bit physical address from 16-bit segment and offset values. Four segment registers manage this:

- CS (Code Segment): holds the base address of the segment containing executable instructions; combined with IP (Instruction Pointer) it forms the address of the next instruction to fetch.
- DS (Data Segment): holds the base address of the segment containing program variables and data, used with offsets for direct/indirect data addressing.
- SS (Stack Segment): holds the base address of the segment used for the stack, combined with SP (Stack Pointer) to track the top of the stack for subroutine calls, returns, and PUSH/POP operations.
- ES (Extra Segment): an additional data segment, often used for string/array operations.

Because these segments can overlap or be placed independently in physical memory, correct execution depends entirely on each segment register being loaded with the correct base value for the process currently running, and on offsets staying within the intended segment's bounds.

Problem-Solving Approach: How Improper Segment Handling Causes the Reported Errors

Incorrect data access: if DS (or ES) is loaded with the wrong segment base — for instance, left pointing at another program's data segment after a context switch, or not reloaded at all — instructions using direct or register-indirect addressing will compute a physical address that lands in the wrong program's memory, silently reading or writing unrelated data.

Stack overflow errors: if SS is set incorrectly, or if the stack segment is sized too small for the depth of nested calls/interrupts a program generates (especially with multiple programs sharing memory), SP can decrement past the start of its allocated segment, corrupting adjacent code or data segments. Failing to balance every PUSH with a corresponding POP, or deep/unbounded recursion, has the same effect within an undersized stack segment.

Multiprogram interference: when several programs execute together without disciplined segment-register reloading at each task switch, one program's CS:IP or SS:SP context can bleed into another's, so execution jumps into or writes over a different program's code, data, or stack — the underlying cause of both symptoms described.

Managing Instruction Execution and Addressing Modes for Correctness

- Always reload all relevant segment registers (CS, DS, SS, ES) explicitly at every task/context switch, rather than relying on previous values, so each program's logical address space maps to its own correct physical region.

- Allocate the stack segment with sufficient size for the expected call depth of each program, and monitor SP to detect underflow/overflow conditions before they corrupt adjacent segments.
- Use based and indexed addressing modes (e.g., via BX/BP as base registers and SI/DI as index registers) with explicit segment-override prefixes where needed, so that offsets are computed transparently within the intended segment rather than assumed.
- Ensure the fetch stage of instruction execution always uses the current CS:IP pair correctly after any jump, call, or interrupt, so control transfers do not inadvertently execute code from a stale or incorrect code segment.

Solution Summary and Justification

Both reported issues trace back to segment registers not being correctly set or bounded for each executing program. Disciplined reloading of CS, DS, SS, and ES at every switch, adequately sized stack segments, and careful use of based/indexed addressing with explicit segment control together prevent cross-program interference, ensuring each program's instructions execute against — and only against — its own intended code, data, and stack regions.

Question 5

A data communication system represents packet information in hexadecimal format for ease of debugging and transmission, but a software engineer mistakenly interprets values, leading to incorrect data processing. Analyze the importance of number system conversions between hexadecimal and binary in such systems, explain how errors in conversion can affect instruction execution, and discuss how efficient CPU design and benchmarking can help detect and minimize such issues in real-time systems.

Understanding the Problem Context

Data communication systems represent packet fields (headers, addresses, checksums) in hexadecimal because it is a compact, human-readable shorthand for the binary values that hardware actually processes. The scenario describes an engineer misinterpreting a hexadecimal value, which then propagates into incorrect data processing — indicating that the accuracy of hex-binary conversion is not merely a cosmetic debugging convenience but a functional requirement for correct system behavior.

Application of Concepts: Importance of Hexadecimal–Binary Conversion

Each hexadecimal digit maps exactly to a 4-bit binary group (nibble), making conversion between the two systems direct and lossless when done correctly — for example, hex value 2F converts to binary 0010 1111. This exact correspondence is why hexadecimal is used: it lets engineers read and write packet data compactly while it still represents the precise binary pattern the CPU and network hardware operate on. Because the CPU ultimately performs all operations — address decoding, checksum computation, control-field interpretation — directly on the binary representation, any human misreading of the hexadecimal shorthand (e.g., transposing digits, or miscounting nibble boundaries) results in a binary value fed to the system that is different from the one intended.

Problem-Solving Approach: How Conversion Errors Affect Instruction Execution

- **Incorrect operand values:** if a mis-converted hex value is loaded as data or as part of an address calculation, the fetch or execute stage of the affected instruction operates on the wrong operand, producing incorrect results without any visible fault.
- **Address/control-field corruption:** packet header fields (such as destination address or length fields) are often given in hex; a conversion slip can cause a system to misinterpret packet boundaries or destinations, leading instructions to read from or write to the wrong memory locations.
- **Undetected propagation:** because the erroneous value is still a syntactically valid number, the instruction executes normally at the hardware level — the error is semantic, not a crash — so it can silently corrupt downstream processing until a checksum or logical mismatch eventually surfaces the fault, by which point multiple packets or computations may already be affected.

Role of Efficient CPU Design and Benchmarking in Detecting and Minimizing Such Issues

CPU design measures: hardware-level parity or checksum verification units, built-in error-detection circuitry (e.g., CRC checks performed in hardware rather than software), and clear separation of address and data buses reduce the chance that a corrupted value silently passes through unnoticed, catching mismatches at the point of data transfer rather than relying solely on manual conversion accuracy.

Benchmarking measures: running systematic test suites and real-time benchmarking with known input–output pairs (including edge-case hex patterns) helps validate that conversion and processing logic behaves correctly under load, exposing conversion-related bugs before deployment. Performance benchmarking under peak traffic conditions also reveals whether error-checking mechanisms introduce acceptable overhead without violating real-time deadlines.

Together, these measures shift error detection from a manual, error-prone human step to an automated, continuously verified part of the system, which is essential for real-time communication systems where a silent conversion error can otherwise propagate uncaught.

Solution Summary and Justification

Since hexadecimal is only a readable proxy for the binary values the CPU actually executes on, any conversion mistake directly injects an incorrect operand into instruction execution. Building in hardware-level error detection (parity/CRC checks) and rigorous benchmarking with known test vectors catches such discrepancies automatically and early, minimizing the risk that a single human conversion error leads to silent, cascading data-processing faults in a real-time system.